

AGT Platform

Chantier 1 — Finalisation & Stabilisation

Conception B5 : Base de connaissance & Features

Phase 0 — Features 13 à 28

Suite du document Features 1–12

Document de référence — Session 32

Gabriel (Lead) — 18 mai 2026

AG Technologies — Confidentiel

Inclut les 3 nouvelles features Banking (F26, F27, F28)

Contents

1	Préambule	2
2	Nouveaux modèles à créer	2
2.1	DetailsFinanciers — Extension produits financiers	2
2.2	ModeleCommunication — KB Communication établissement	2
2.3	Annonce — ResultRecord Communication	3
2.4	ScenarioProspection — KB capture_prospect	3
2.5	DemandeConciergerie — ResultRecord Conciergerie	3
2.6	EmailLog — Log universel des emails	4
2.7	RapportClient — Analyse globale d'un contact	4
2.8	ContactNote — Notes manuelles	4
3	Modifications de modèles existants	5
4	Règles métier transversales (nouvelles)	6
4.1	Distinction triptyque CRM	6
4.2	Bot M2M agences	6
4.3	Garde-fous banking	6
4.4	Autres garde-fous critiques	6
4.5	Renommage slug	6
4.6	Seeder banking centralisé	6
5	Fiches features 13 à 28	7
6	Tableau récapitulatif des actions	11
7	Features hors scope — résumé	12

1 Préambule

Ce document est la suite directe du document Features 1–12. Il couvre les features 13 à 28, incluant **3 nouvelles features Banking** ajoutées en Chantier 1.

Features hors scope Chantier 1 : `commande_paiement · paiement_en_ligne · agent_vocal · dashboard` (couvert par B6 — Penka)

2 Nouveaux modèles à créer

2.1 DetailsFinanciers — Extension produits financiers

One-to-one optionnel sur `ItemCatalogue`. Ne pollue pas les items non-banking.

```
GRAND_TYPE = [compte, credit, epargne, carte, transfert]

class DetailsFinanciers(models.Model):
    item = OneToOneField(ItemCatalogue, related_name="
        details_financiers")
    grand_type = CharField(50, choices=GRAND_TYPE, default="autre")
    sous_type = CharField(100) blank=True # libre : "Cr dit PME",
        "Visa Classic"
    taux_annuel_min = DecimalField(5,2) null=True
    taux_annuel_max = DecimalField(5,2) null=True
    taux_type = CharField(choices=[fixe, variable, negocie]
        default=fixe
    montant_min = DecimalField(15,2) null=True
    montant_max = DecimalField(15,2) null=True
    devise = CharField(10) default="XAF"
    duree_min_mois = PositiveIntegerField null=True
    duree_max_mois = PositiveIntegerField null=True
    documents_requis = JSONField default=[]
    delai_traitement = CharField(100) blank=True
    conditions_specifiques = JSONField default={}
    mis_a_jour_le = DateTimeField auto_now=True
```

Grand type	Slug	Exemples
Comptes	<code>compte</code>	Courant, Épargne, Jeunes, Entreprise
Crédits	<code>credit</code>	Personnel, PME, Immobilier, Scolaire
Épargne & Placement	<code>epargne</code>	Livret, Plan épargne, Assurance vie
Cartes	<code>carte</code>	Visa, Mastercard, Prépayée, Débit
Transferts & Services	<code>transfert</code>	Mobile money, Virement international

2.2 ModeleCommunication — KB Communication établissement

```
TYPE = [annonce_generale, resultats_scolaires, absence, evenement,
        convocation, urgence, rappel_echeance, autre]

class ModeleCommunication(models.Model):
    entreprise = FK(Entreprise)
    type_communication = CharField(choices=TYPE)
    titre_template = CharField(200)
```

```

corps_template      = TextField    # avec variables {nom_etudiant},
    {date}...
variables_disponibles = JSONField list default=[]
is_active           = BooleanField default=True
secteur             = FK(SecteurActivite, null=True)  # filtre par
    secteur

```

2.3 Annonce — ResultRecord Communication

```

class Annonce(models.Model):
    STATUT = [brouillon, planifiee, envoyee, echouee]
    entreprise      = FK(Entreprise)
    modele          = FK(ModeleCommunication, null=True)
    sujet           = CharField(300)
    corps           = TextField
    statut          = CharField choices=STATUT  default=brouillon
    date_envoi_planifiee = DateTimeField null=True
    date_envoi_effectif = DateTimeField null=True
    destinataires_count = PositiveIntegerField default=0
    destinataires_filtre = JSONField default={}
    envoye_par      = FK(User, null=True)
    source          = CharField choices=[admin, bot]  default=admin
    created_at      = DateTimeField auto_now_add=True

```

2.4 ScenarioProspection — KB capture_prospect

```

class ScenarioProspection(models.Model):
    DECLENCHEUR = [premier_message, mot_cle, toujours]
    entreprise  = FK(Entreprise)
    nom         = CharField(150)
    questions   = JSONField list
    # [{"ordre": 1, "question": "Quel est votre besoin ?",
    #   "champ_cible": "interet_principal"}]
    declencheur = CharField choices=DECLENCHEUR  default=premier_message
    is_active   = BooleanField default=True

```

2.5 DemandeConciergerie — ResultRecord Conciergerie

```

class DemandeConciergerie(models.Model):
    STATUT = [recue, prise_en_charge, en_cours, effectuee, annulee]
    contact      = FK(Contact)
    conversation  = FK(AIConversation)
    agence       = FK(Agence)
    service      = FK(ItemCatalogue)
    chambre      = CharField(20) blank=True
    heure_souhaitee = DateTimeField null=True
    notes_client = TextField blank=True
    statut       = CharField choices=STATUT  default=recue
    pris_en_charge_par = FK(User, null=True)
    effectuee_le = DateTimeField null=True
    source       = "bot"
    created_at   = DateTimeField auto_now_add=True

```

2.6 EmailLog — Log universel des emails

Créer dans `apps/notifications/`. Remplace le stub actuel de `send_email`. L'entreprise peut lire chaque email envoyé avec son contenu complet.

```
SOURCE = [reservation, commande, inscription, dossier,
           annonce, transfert, conciergerie, rappel, autre]

class EmailLog(models.Model):
    entreprise = FK(Entreprise)
    destinataire = EmailField
    sujet = CharField(300)
    corps = TextField # contenu complet lisible par l'
                    entreprise
    template = CharField(100) blank=True
    statut = CharField choices=[en_attente, envoye, echec]
              default=en_attente
    envoye_le = DateTimeField null=True
    erreur = TextField blank=True
    source_type = CharField choices=SOURCE blank=True
    source_id = UUIDField null=True
    created_at = DateTimeField auto_now_add=True
```

2.7 RapportClient — Analyse globale d'un contact

```
class RapportClient(models.Model):
    contact = FK(Contact, related_name="rapports")
    resume_global = TextField
    points_forts = JSONField list default=[]
    points_attention = JSONField list default=[]
    recommandations = JSONField list default=[]
    nb_conversations = PositiveIntegerField
    periode_debut = DateField
    periode_fin = DateField
    genere_le = DateTimeField auto_now_add=True
    genere_par = CharField choices=[bot, admin] default=bot
```

2.8 ContactNote — Notes manuelles

```
class ContactNote(models.Model):
    contact = FK(Contact, related_name="notes")
    texte = TextField
    auteur = FK(User)
    created_at = DateTimeField auto_now_add=True
```

3 Modifications de modèles existants

Modèle	Champ	Description
Bot	agences M2M	Remplace <code>agence</code> FK — un bot peut servir N agences
Bot	<code>rappel_avant_heures</code>	Délai rappel email (défaut 24h)
Contact	<code>statut_crm</code>	Ajouter valeur <code>contact</code> (entre prospect et client)
Contact	<code>tags</code>	JSONField list — labels personnalisés
Contact	<code>resume_ia</code>	Dernier résumé généré par le bot
AIConversation	<code>rapport_texte</code>	Résumé généré en fin de conversation
AIConversation	<code>rapport_genere_le</code>	Date de génération du rapport
AIConversation	<code>points_cles</code>	JSONField list — points importants détectés
TacheRelance	<code>statut</code>	<code>en_attente</code> / <code>executee</code> / <code>echouee</code>
TacheRelance	<code>executee_le</code>	DateTimeField null=True
ProspectCapture	<code>score_qualification</code>	Score 0-100 calculé sur les réponses
ProspectCapture	<code>notes_qualification</code>	Résumé bot des réponses collectées
ProspectCapture	<code>scenario</code> FK	Lien vers <code>ScenarioProspection</code> utilisé
Inscription	<code>statut</code>	Ajouter valeur <code>documents_manquants</code>
Inscription	<code>notes_dossier</code>	Notes internes (cohérence avec Dossier)
ProgrammeAdmission	<code>niveau</code>	Enrichir : <code>primaire</code> , <code>college</code> , <code>lycee</code> , <code>bts_dut...</code>
ProgrammeAdmission	<code>frais_scolarite_annuels</code>	Frais annuels
ProgrammeAdmission	<code>places_disponibles</code>	Nombre de places
ProgrammeAdmission	<code>documents_requis</code>	JSONField list
ProgrammeAdmission	<code>etapes_inscription</code>	JSONField list — étapes ordonnées
ServiceCitoyen	<code>etapes_procedure</code>	JSONField list — étapes démarche
ServiceCitoyen	<code>horaires_service</code>	JSONField — quand ce bureau est ouvert
ServiceCitoyen	<code>delai_relance_jours</code>	Après combien de jours re-notifier (défaut 7)
Dossier	<code>service_citoyen</code> FK	Lien vers le service demandé
SpecialiteMedicale	<code>mots_cles_symptomes</code>	JSONField list — pour l'orientation
SpecialiteMedicale	<code>contact_urgence</code>	Numéro d'urgence du service
ProfilEntreprise	<code>contact_urgence_global</code>	Numéro urgences générales
Payment	<code>reservation</code> FK	Lien paiement réservation (null=True)
ItemCatalogue	<code>action_suivante</code>	Ajouter valeur <code>traiter_demande</code>

4 Règles métier transversales (nouvelles)

4.1 Distinction triptyque CRM

Feature	Cible	Direction	Canal
<code>capture_prospect</code>	Inconnus / 1er contact	Entrant	WhatsApp uniquement
<code>gestion_crm</code>	Contacts connus	Consultation	Interface admin
<code>communication_etablissement</code>	Contacts connus	Sortant	Email uniquement

4.2 Bot M2M agences

Un bot peut servir 1 à N agences. Si 1 agence : sélection automatique, pas de question. Si N agences : le bot demande la ville en début de conversation. `AIConversation.agence` est renseigné dès la sélection.

4.3 Garde-fous banking

BUG B-001 BLOQUANT : Le bot ne donne jamais le solde d'un compte.
BUG B-002 BLOQUANT : Le bot ne fait jamais de recommandation financière personnalisée.
 Ces règles sont dans la description système du Bot banking.

4.4 Autres garde-fous critiques

Santé : jamais de diagnostic → oriente vers médecin. Urgence → `contact_urgence` immédiat.

Éducation : jamais de résultats individuels → redirection officielle. Communications = email uniquement.

Public : le bot représente une seule entité → hors périmètre = TransfertHumain.

4.5 Renommage slug

`conversion_prospects` → `capture_prospect`
 Impact : backend seeder + actions CRM + frontend `SECTOR_THEMES.defaultFeatures`

4.6 Seeder banking centralisé

`seed_bank.py` est isolé — non appelé par `python manage.py seed`. Action : intégrer dans `SEEDERS_REGISTRY` sous clé "banking".

5 Fiches features 13 à 28

Feature 13 — catalogue_produits_financiers

Description : Une banque publie ses produits. Le bot présente et oriente vers un RDV conseiller — sans jamais donner de conseil personnalisé.

Booléens :

```
entreprise_configure_kb=True | bot_lit_kb=True |
bot_ecrit_result=False (AIActionLog) | entreprise_peut_ecrire_result=False
```

Actions à mener :

1. Créer `DetailsFinanciers` (§2.1) — OneToOne sur `ItemCatalogue`
2. Migrer `ProduitFinancier` (S2) → `ItemCatalogue` + `DetailsFinanciers`
3. `action_suivante` = "proposer_rdv" par défaut pour tous les produits
4. Intégrer `seed_bank.py` dans `SEEDERS_REGISTRY`
5. Vérifier garde-fous B-001 et B-002 dans description Bot banking

Feature 14 — inscription_admission

Description : Un étudiant dépose une demande d'admission. Le bot guide, collecte les pièces, crée le dossier. L'établissement traite depuis son dashboard. **Tous niveaux :** primaire, collège, lycée, BTS, licence, master, doctorat, formation professionnelle.

Booléens :

```
entreprise_configure_kb=True | bot_lit_kb=True |
bot_ecrit_result=True | entreprise_peut_ecrire_result=True
```

Actions à mener :

1. Enrichir `ProgrammeAdmission` : 8 niveaux, 'frais_scolarite_annuels', 'places_disponibles', 'documents_requis', 'etapes_inscription'
2. Ajouter sur `Inscription` : 'numero_dossier', 'programme FK', 'est_hors_pperiode', 'notes_dossier', statut documents_manquants
3. Créer action `get_programmes(niveau?, filiere?)`
4. Règle hors période : créer + signaler, jamais refuser (R14 éducation)
5. Email uniquement, jamais WhatsApp proactif

Feature 15 — orientation_patient

Description : Le patient décrit ses symptômes. Le bot oriente vers la bonne spécialité et prend le RDV — sans jamais faire de diagnostic.

Actions à mener :

1. Ajouter sur `SpecialiteMedicale` : 'mots_cles_symptomes', 'contact_urgence', convention 'ressource_id' dans JSON medecins
2. Ajouter 'contact_urgence_global' sur `ProfilEntreprise`
3. Créer actions `get_specialites()` et `get_specialite_par_symptomes(symptoms)`
4. Disponibilités médecins : hors scope C1 (créneaux généraux via Ressource)

Feature 16 — orientation_citoyens

Description : Un citoyen s'informe sur les démarches administratives et ouvre son dossier via WhatsApp. L'agent traite depuis son interface.

Actions à mener :

1. Ajouter sur `ServiceCitoyen` : 'etapes_procedure', 'horaires_service', 'delai_relance_jours'
2. Ajouter 'service_citoyen FK' sur `Dossier`
3. Créer action `get_services_publics(categorie?)`

Feature 17 — multi_agences

Description : Un bot peut servir une ou plusieurs agences (configurable). Le bot demande au client son agence si nécessaire.

Actions à mener :

1. Migrer `Bot.agence FK` → `Bot.agences M2M` (agence actuelle = premier élément du M2M)
2. Ajouter sur `Agence` : 'latitude', 'longitude', 'description_courte'
3. Créer action `get_agences(ville?)`
4. Implémenter logique de sélection agence dans `AIConversation`

Hors scope C1 : routing centralisé (un numéro → N bots)

Feature 18 — gestion_crm

Description : Chaque interaction enrichit automatiquement la fiche client. L'entreprise dispose d'un CRM vivant alimenté par le bot.

Actions à mener :

1. Enrichir `Contact` : statut contact, 'tags', 'resume_ia'
2. Créer modèles `ContactNote` et `RapportClient`
3. Enrichir `AIConversation` : 'rapport_texte', 'rapport_genere_le', 'points_cles'
4. Page `/contacts` : statuts CRM, tags, notes, timeline, score

Feature 19 — conciergerie

Description : Un client hôtel demande un service (taxi, room service, blanchisserie). Le bot enregistre, le staff exécute.

Actions à mener :

1. Créer tab "Conciergerie" dans `/knowledge` → `ItemCatalogue`
2. Créer modèle `DemandeConciergerie` (§2.5)
3. Créer action `get_services_conciergerie(type?)`
4. Ajouter `traiter_demande` dans `action_suivante` choices

Feature 20 — communication_etablissement

Description : L'établissement envoie des communications groupées par email. Le bot répond aux questions sur les annonces et peut déclencher des envois. **Étendu à tous les secteurs** (`is_default=False`).

Règle absolue : email uniquement, jamais WhatsApp proactif.

Actions à mener :

1. Créer `ModeleCommunication` (§2.2) + `Annonce` (§2.3)
2. Créer actions `get_annonces (type?)` et `create_annonce (...)`
3. Ajouter à `SECTOR_MATRIX` pour tous les secteurs (`is_default=False`)
4. Route frontend `/modules/communications`

Feature 21 — `capture_prospect` (ex `conversion_prospects`)

Description : Quand un inconnu écrit pour la première fois, le bot le qualifie et capture ses coordonnées. Point d'entrée : inbound WhatsApp uniquement. **Étendu à tous les secteurs** (`is_default=False`).

Actions à mener :

1. **Renommer** `conversion_prospects` → `capture_prospect` partout
2. Créer `ScenarioProspection` (§2.4)
3. Enrichir `ProspectCapture` : `'score_qualification'`, `'notes_qualification'`, `'scenario FK'`
4. Ajouter à `SECTOR_MATRIX` pour tous les secteurs

Feature 22 — `paiement_en_ligne` — HORS SCOPE CHANTIER 1

Accord commercial providers requis (Orange Money, MTN MoMo). Badge "Bientôt disponible". Infrastructure `apps/payments/` conservée sans extension.

Feature 23 — `dashboard` — Couvert par B6

V1 fonctionnelle. Refonte 3 niveaux documentée dans `AGT_Chantier1.pdf` — assignée à Penka. Dépend de B5 terminé.

Feature 24 — `agent_vocal` — HORS SCOPE CHANTIER 1

Architecture : STT → AI Agent existant → TTS. Même agent, nouveau canal. `canal="vocal"` déjà dans `AIConversation`. Chantier 2 : pipeline STT/TTS.

Feature 25 — `emails_rappel`

Description : Le bot programme automatiquement des emails de rappel après réservations et commandes. L'entreprise voit chaque email envoyé avec son contenu.

12 scénarios couverts : confirmation réservation · rappel RDV · changement statut réservation · confirmation commande · changement statut commande · inscription soumise · résultat admission · documents manquants · dossier traité · communication établissement · alerte transfert · alerte conciergerie.

Actions à mener :

1. Créer `EmailLog` (§2.6) — log complet, contenu lisible
2. Refactoriser `send_email` → crée `EmailLog` à chaque appel
3. Enrichir `TacheRelance` : `'statut'` + `'executée_le'`
4. Ajouter `'rappel_avant_heures'` sur Bot (défaut 24h)
5. Vérifier tâche Celery planifiée
6. Page `/modules/emails` — liste + contenu lisible par l'entreprise
7. Configurer SMTP production

Nouvelles features Banking — Chantier 1

Features 26, 27 et 28 ajoutées pour le secteur Banque & Microfinance. Clients banking = parmi les plus grands clients AGT. Ces features sont prioritaires.

Feature 26 — `simulation_credit` (BANKING — NOUVEAU C1)

Description : Un client demande une simulation de crédit. Le bot calcule mensualités et coût total — sans accéder aux données personnelles.

Formule (calcul côté serveur, jamais LLM) :

```
taux_mensuel = taux_annuel / 12 / 100
mensualite   = (montant * taux_mensuel) / (1 - (1 + taux_mensuel)
               ^(-duree_mois))
cout_total   = mensualite * duree_mois
cout_interets = cout_total - montant
```

Actions à mener :

1. Créer action `simulate_credit(montant, duree_mois, taux_annuel?)`
2. Si taux non fourni → utilise `DetailsFinanciers.taux_annuel_min` du produit mentionné
3. Logger dans `AIActionLog` — pas de `ResultRecord` (calcul informatif)
4. Disclaimer obligatoire : "simulation indicative — consultez un conseiller"

Feature 27 — `suivi_dossier` (BANKING — NOUVEAU C1)

Description : Un client suit l'état de sa demande (ouverture compte, crédit) sans se déplacer. Le conseiller traite depuis le dashboard.

Réutilise `Dossier` (apps/dossiers) — types banking : `ouverture_compte` · `demande_credit` · `renouvellement_carte` · `changement_conseiller` · `reclamation` · `autre`

Actions à mener :

1. Créer action `get_dossier_statut(numero_dossier)` avec vérification `Dossier.contact.phone == conversation.contact.phone`
2. Créer action `create_dossier_banking(type_demande)` — guide le client
3. Format numéro : BNK-YYYYMMDD-XXXX
4. Notifications email à chaque changement de statut

Feature 28 — `collecte_documents` (BANKING — NOUVEAU C1)

Description : Le bot guide le client étape par étape dans les documents à fournir. L'entreprise valide manuellement la réception de chaque pièce.

Ce qui existe : `documents_requis` et `documents_fournis` sur `Dossier` (JSONB) + `DetailsFinanciers.documents_requis` (JSONField).

Actions à mener :

1. Créer action `list_documents_requis(produit_ou_type_dossier)`
2. Créer action `confirmer_document(dossier_id, document_nom)` — ajoute dans `Dossier.documents_fournis`
3. Interface entreprise : cases à cocher par pièce (validation manuelle)
4. Si tous les documents fournis → notifie le conseiller

Hors scope C1 : upload réel de fichiers (C2 — stockage cloud).

6 Tableau récapitulatif des actions

Priorité	Action	Feature(s)	Type
Haute	Renommer <code>conversion_prospects</code> → <code>capture_prospect</code>	<code>capture_prospect</code>	Config
Haute	Intégrer <code>seed_bank</code> dans <code>SEEDERS_REGISTRY</code>	<code>banking</code>	Seeder
Haute	Créer <code>DetailsFinanciers</code> + migrer <code>ProduitFinancier</code>	F13	Migration
Haute	Créer <code>EmailLog</code> + refactoriser <code>send_email</code>	<code>emails_rappel</code>	Backend
Haute	Migrer <code>Bot.agence FK</code> → <code>Bot.agences M2M</code>	<code>multi_agences</code>	Migration
Moyenne	Créer <code>DemandeConciergerie</code>	<code>conciergerie</code>	Backend
Moyenne	Créer <code>ModeleCommunication</code> + <code>Annonce</code>	F20	Backend
Moyenne	Créer <code>ScenarioProspection</code> + enrichir <code>ProspectCapture</code>	F21	Backend
Moyenne	Enrichir <code>ProgrammeAdmission</code> (7 champs)	F14	Migration
Moyenne	Enrichir <code>SpecialiteMedicale</code> + <code>ProfilEntreprise</code>	F15	Migration
Moyenne	Enrichir <code>ServiceCitoyen</code> + <code>Dossier</code>	F16	Migration
Moyenne	Créer <code>RapportClient</code> + <code>ContactNote</code>	<code>gestion_crm</code>	Backend
Moyenne	Enrichir <code>Contact</code> (statut, tags, <code>resume_ia</code>)	<code>gestion_crm</code>	Migration
Moyenne	Actions <code>banking</code> (<code>simulate_credit</code> , <code>dossier</code>)	F26, F27, F28	Backend
Basse	<code>communication_etablissement</code> → tous secteurs	F20	Seeder
Basse	<code>capture_prospect</code> → tous secteurs	F21	Seeder
Basse	Pages <code>/modules/emails</code> et <code>/modules/communications</code>	F25, F20	Frontend
Basse	Config SMTP production	<code>emails_rappel</code>	Infra

7 Features hors scope — résumé

Feature	Chantier	Raison
<code>commande_paiement</code>	C2	Provider paiement requis
<code>paiement_en_ligne</code>	C2	Accord commercial Orange Money / MTN
<code>agent_vocal</code>	C2	Pipeline STT/TTS — même agent, nouveau canal
<code>dashboard</code>	B6	Couvert par B6 (Penka) — dépend de B5
Upload documents	C2	Infrastructure stockage cloud
Livraison (commandes)	C2	Module à part entière
Routing bot centralisé	C2	Un numéro → N bots
Agendas médicaux fins	C2	Gestion individuelle praticiens
